



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

Parallel Programming - Parallel Algorithms -

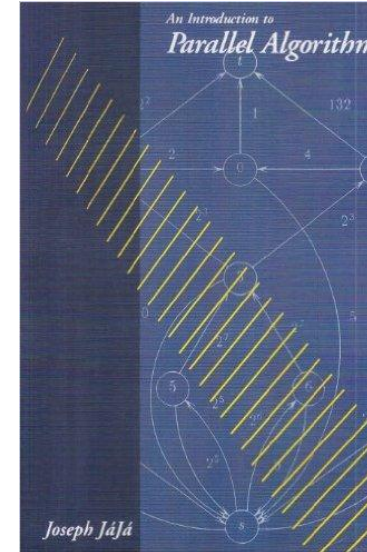
Prof. Dr. Felix Wolf
Technical University of Darmstadt

Outline

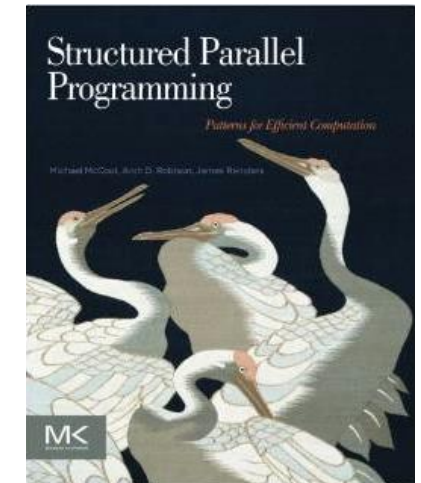
- Parallel processing
- Basic parallel models
- Performance of parallel algorithms
- Work-depth model
- Glimpse of BSP & LogP

Literature

- An Introduction to Parallel Algorithms
Joseph JáJá, Addison Wesley, 1992



- Structured Parallel Programming
Michael McCool, Arch D. Robison, James Reinders, Morgan Kaufmann, 2012



Objectives of parallel processing



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

Faster
solutions



Solving
larger
problems



Less
energy to
solution



More processors
but at a lower
frequency



Speedup

- Assume problem **P** with input size **n**
- Sequential complexity of P is **$T^*(n)$**
- Let **A** be a parallel algorithm that solves P in time **$T_p(n)$**

Best sequential
algorithm solves P
within this time

$$\text{Speedup: } S_p(n) = \frac{T^*(n)}{T_p(n)}$$



Speedup (2)

- Desirable are algorithms with $S_p(n) \approx p$

Return
proportional to
investment

- Note that $T^*(n) \leq T_1(n)$
- Common practice to replace $T^*(n)$ with time bound of the **best known** sequential algorithm



Parallel efficiency

$$E_p(n) = \frac{T_1(n)}{p * T_p(n)}$$

- Metric for cost of parallelization (e.g., communication)
- Without super-linear speedup

Example:

$$T_1(n) := 4$$

$$T_p(n) := 1.25$$

$$p := 4$$

$$E_4(n) = 0.8$$

$$E_p(n) \leq 1$$

Super-linear speedup possible

- Critical data structures may fit into the aggregate cache



Parallel efficiency (2)

- Lower bound on running time $T_p(n) \geq T_\infty(n)$

Influenced by
sequential part or
“grain size”

- Upper bound on efficiency $E_p(n) \leq \frac{T_1(n)}{p * T_\infty(n)}$

- Efficiency drops quickly as p increases beyond $\frac{T_1(n)}{T_\infty(n)}$

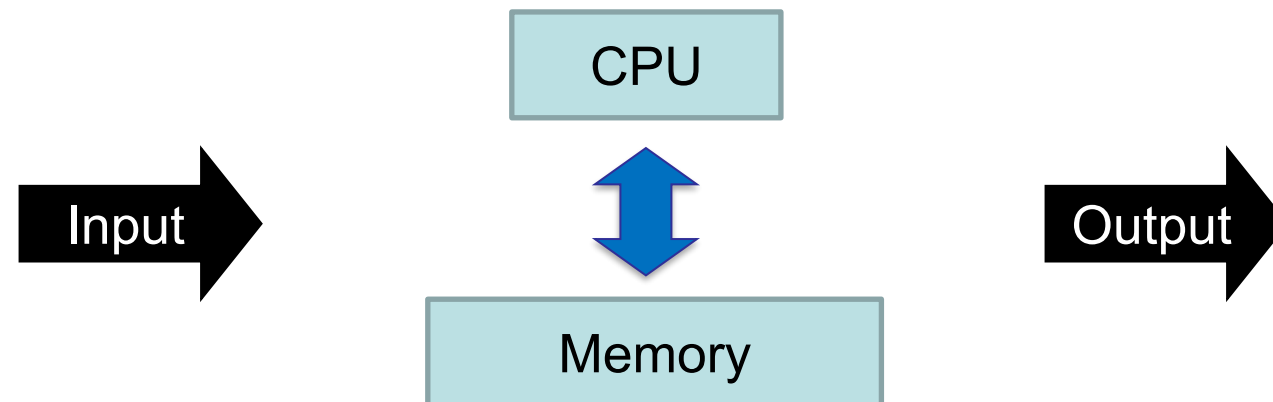
Bounds on resources (required by sequential algorithm)

Requirements (e.g., time or space) specified as a function of the input size

- $T(n) = O(f(n))$ if there exist positive constants c and n_0 such that $T(n) \leq c \cdot f(n)$, for all $n \geq n_0$
- $T(n) = \Omega(f(n))$ if there exist positive constants c and n_0 such that $T(n) \geq c \cdot f(n)$, for all $n \geq n_0$
- $T(n) = \theta(f(n))$ if $T = O(f(n))$ and $T(n) = \Omega(f(n))$

(Sequential) running time

- Specified in terms of the number of basic operations
 - Reading to memory
 - Writing to memory
 - Arithmetic and logical operations
- Underlying computational model = **random access machine (RAM)**



Simple language for expressing algorithms



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

variable := expression

Assignment

exit

```
begin
  statement
  statement
  [...]
  statement
end
```

Sequence of
statements

Termination

Conditional
statement

Loops

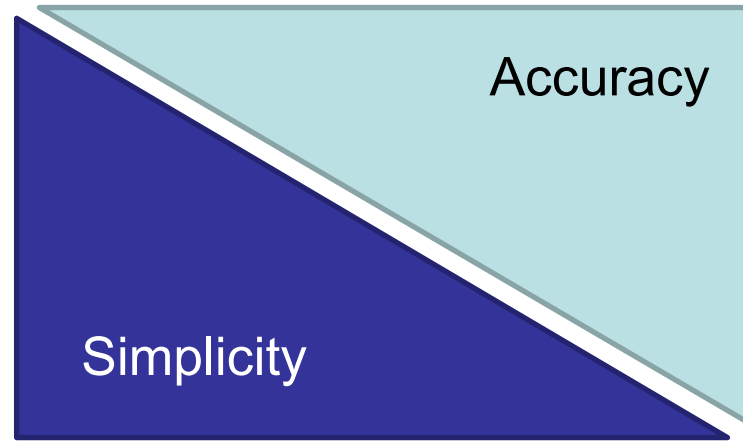
```
if (condition) then statement [else statement]
```

```
for variable = initial val. to final val. do statement
```

```
while (condition) do statement
```

Parallel models

- Realistic predictions
- Actual implementations



- Ease of algorithm design and analysis
- Portability
- General insights

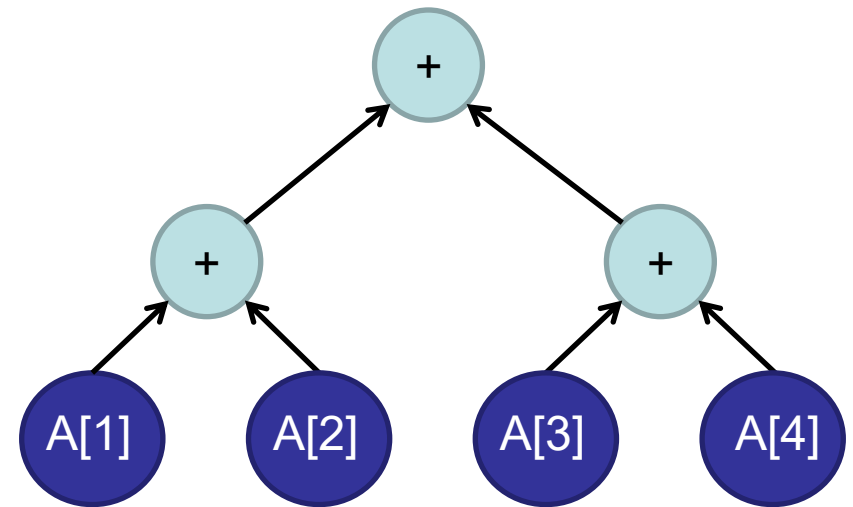
Parallel models

- Directed acyclic graph
- Shared-memory model
- Network model
- Work–depth model

Directed acyclic graph (DAG)

Many computations can be represented by DAGs, e.g., $\sum_{i=1}^4 A[i]$

- Inputs: nodes with no incoming arcs;
input size = number of input nodes
- Operations: inner nodes (max. indegree = 2);
each operation takes one time unit
- Outputs: nodes with outdegree zero
- No branches



DAG (2)

- Algorithm represented by family of DAGs $\{G_n\}$
 - G_n is the algorithm for input size n
- Suitable for numeric computations
 - Branches mostly in loops
 - Can be resolved via loop unrolling
- Focus on data dependences
- Completely architecture-independent



DAG scheduling

- Assumption: p processors & any processor can access the data computed by any other processor without additional cost

Schedule means here processor $j_i \leq p$ executes node i at time unit t_i

- A schedule associates with each internal node i a pair (j_i, t_i) such that
 - If $t_i = t_k$ for some nodes $i \neq k$, then $j_i \neq j_k$. That is, each processor can perform only a single operation during each time unit
 - If (i, k) is an arc in the graph, then $t_k \geq t_i + 1$. That is, the operation represented by node k should be scheduled after the operation represented by node i has been completed



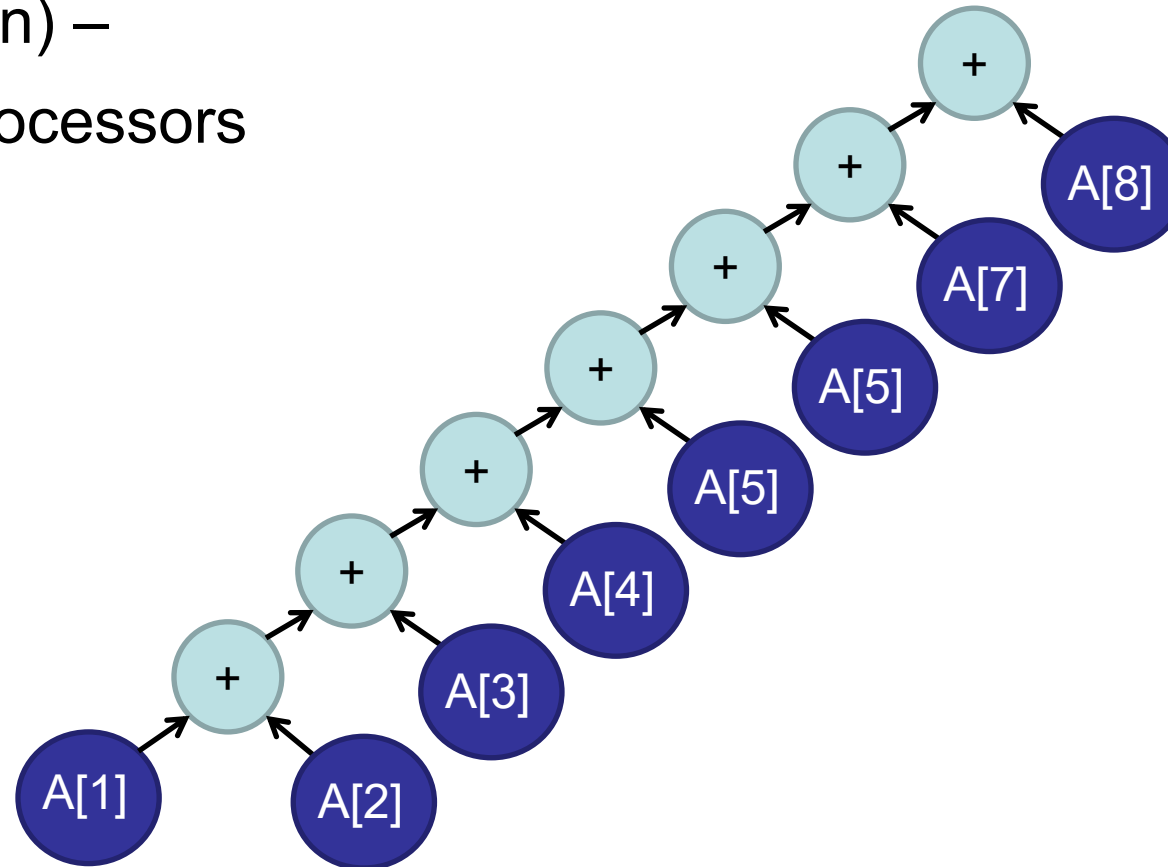
DAG scheduling (2)

- Sequence $\{(j_i, t_i) \mid i \in N\}$ is called **schedule** with
 - N being the set of nodes
 - j_i being a processor & t_i a time
- Time for executing the schedule is given by $\max_{i \in N}(t_i)$
- The **parallel complexity** of the DAG is $T_p(n) = \min_{i \in N} \{\max(t_i)\}$
 - Minimum across all schedules that use p processors
- Depth of the DAG = longest path between input and output
 - Lower bound on $T_p(n)$ for any number of processors

Example – sum reduction (sequence)

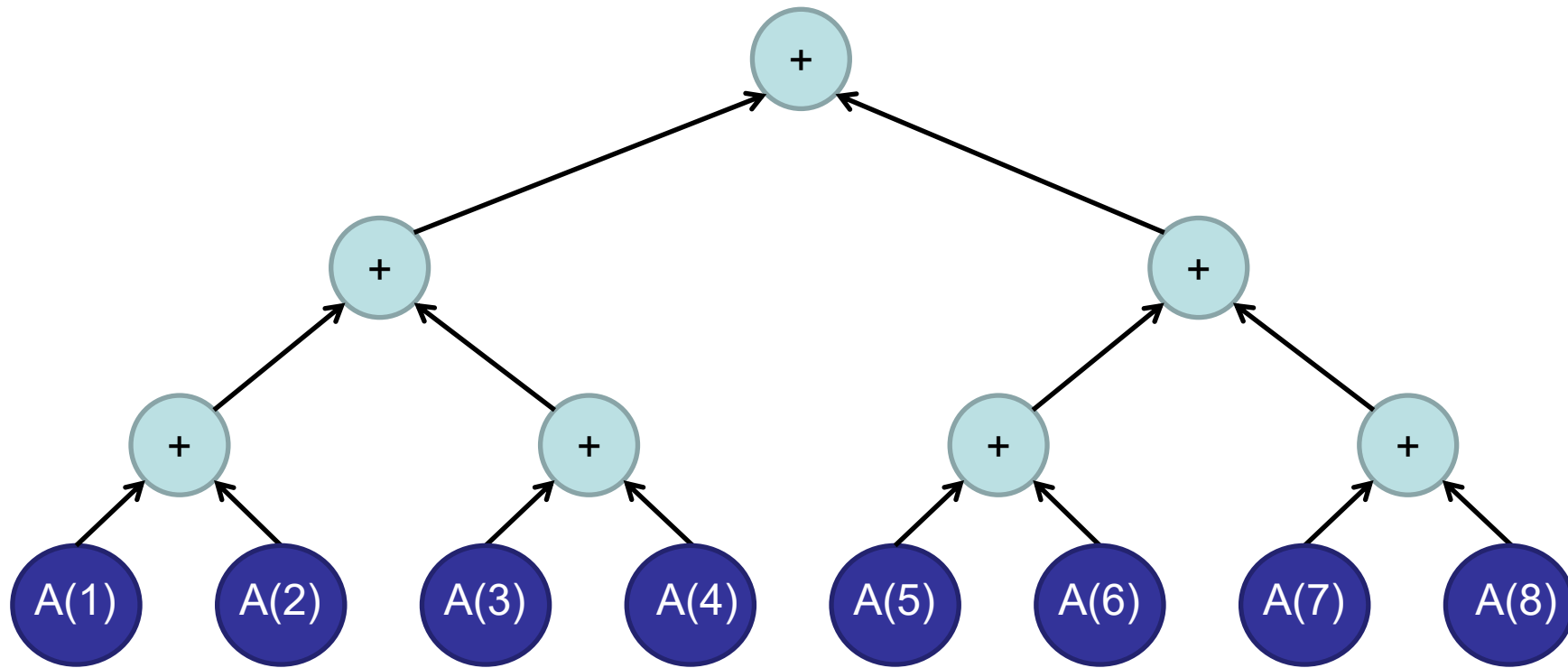


Best schedule takes $O(n)$ –
no matter how many processors



Example – sum reduction (tree)

Best schedule takes $O(\log n)$ with $n/2$ processors

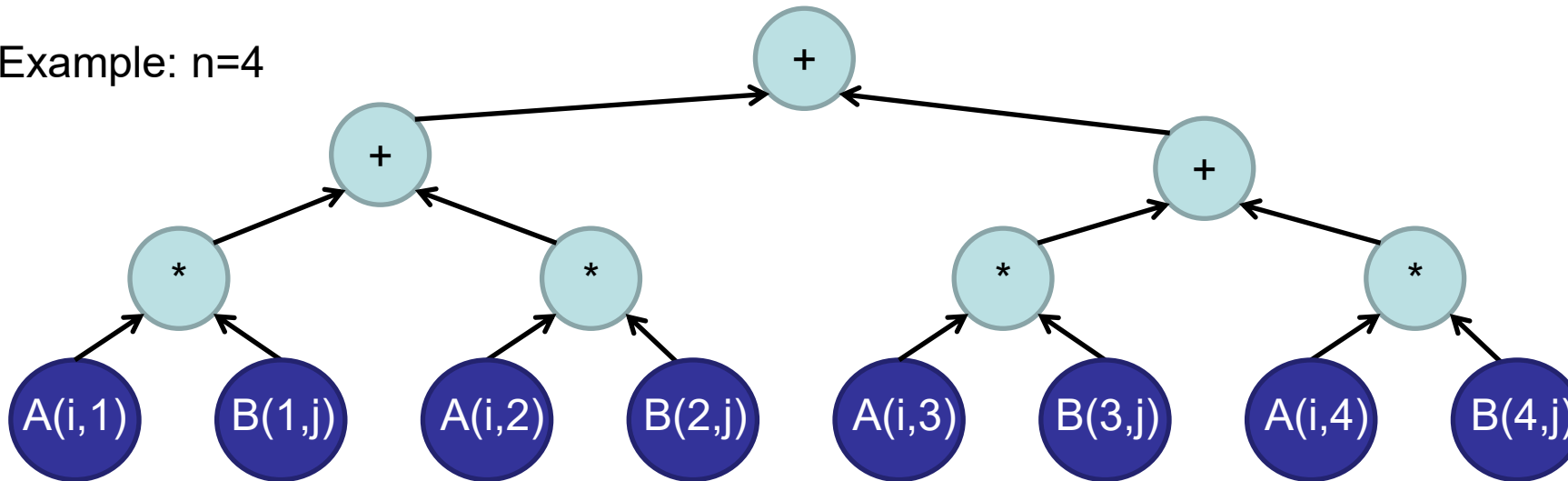


Example – matrix–matrix multiplication

$C = AB$ with A and B being two $n \times n$ matrices

- Standard algorithm $C(i, j) = \sum_{k=1, \dots, n} A(i, k)B(k, j)$

Example: $n=4$



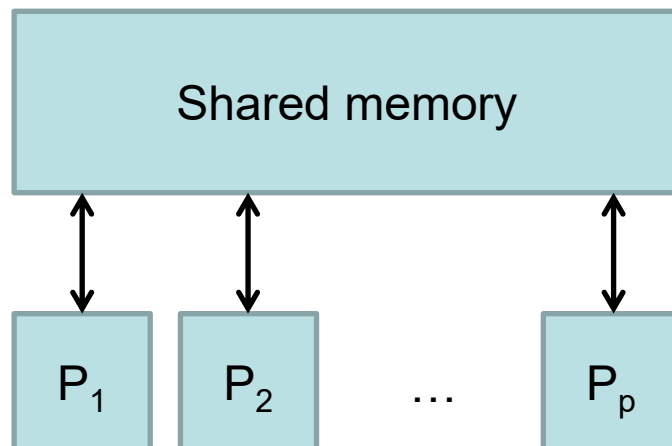
Given n^3 processors, DAG can be scheduled in $O(\log n)$ time

Shared-memory model

Natural extension of the basic sequential model

- Each processor has local memory
- Each processor can execute its own local program
- Communication via shared memory (aka global memory)

MIMD



Processor ID known locally. Can be referred to in program.

Modes of operation

Synchronous

- All processor operate under control of common clock
- Also known as **parallel random-access machine** (PRAM)
[Fortune and Wyllie, 1978]

Asynchronous

- Each processor operates under a separate clock
- Programmer is responsible for
 - Setting appropriate synchronization points
 - Ensuring that the correct value of a variable is obtained

Example – matrix–vector multiplication

- Let **A** be an $n \times n$ matrix, let **x** be vector of order **n**, both stored in shared memory
- Assume we have $p \leq n$ processors such that $r = n / p$ is an integer
- A is partitioned as follows

$$A = \begin{bmatrix} \square & A_1 & \square \\ \square & & \square \\ \square & A_2 & \square \\ \square & \vdots & \square \\ \square & & \square \\ \square & A_p & \square \end{bmatrix}$$

Simplifies the
following slides

- Mode of operation is asynchronous

Algorithm – matrix–vector multiplication

Input

- A , x in shared memory
- order n , processor number i , number of processors p in local memory

Output

- Components $(i-1)r + 1, \dots, ir$ of $y = Ax$ in shared variable y

`global_read(global_src, local_dest)`
`global_write(local_src, global_dest)`

begin

1. `global_read(x,z)`
2. `global_read(A((i-1)r + 1 : ir, 1:n), B)`
3. Compute $w = Bz$
4. `global_write(w, y((i-1)r + 1 : ir))`

end

Discussion

- Concurrent read of x – but no data race
- Complexity
 - Computation: $O(n^2/p)$
 - Communication: $O(n^2/p)$ read; $O(n/p)$ write

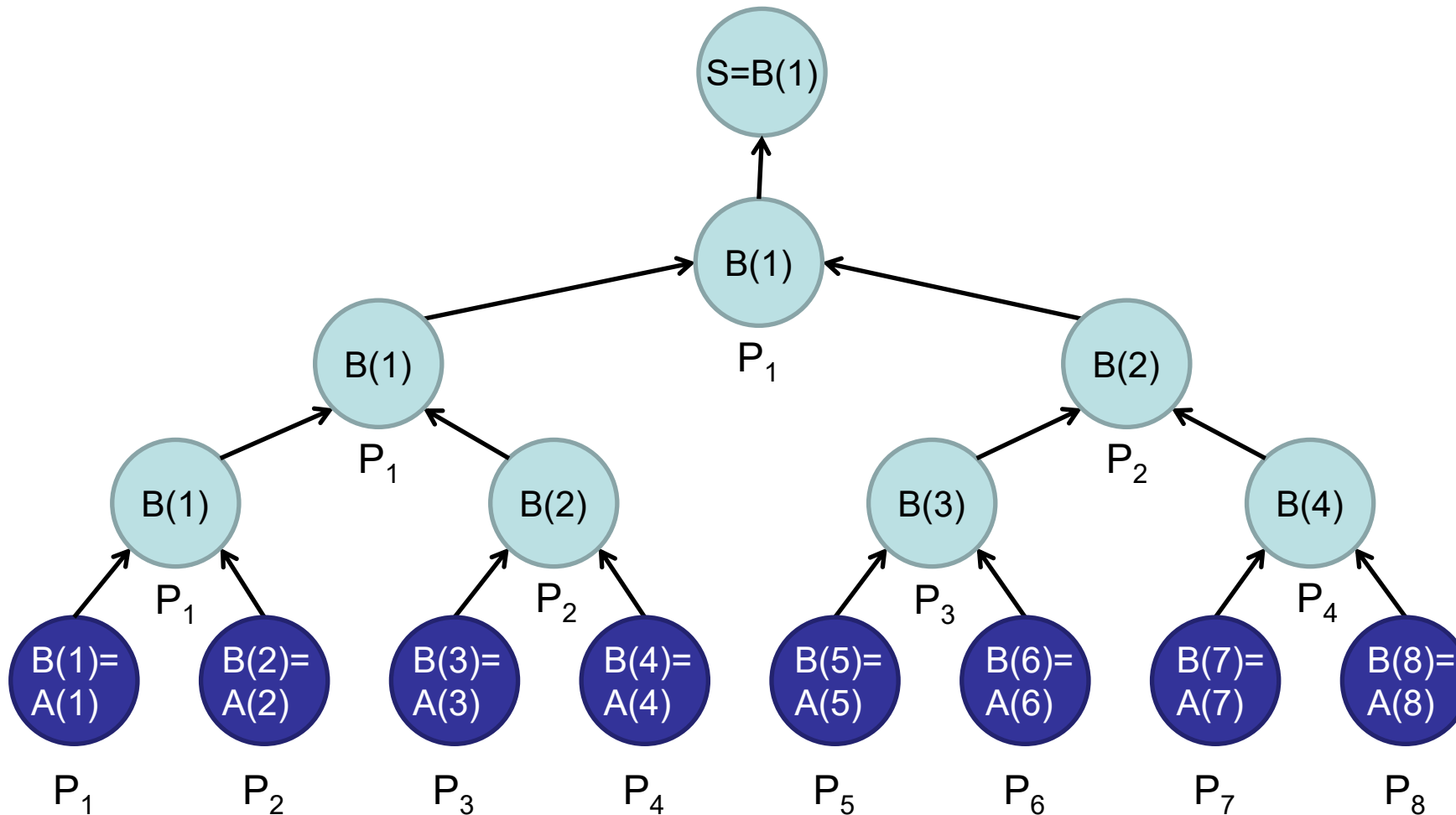


Sum on the PRAM

- **Input** – array A of order $n=2^k$ in SM;
processor number i
- **Output** – sum of entries S in SM

```
begin
  1. global_read(A(i),a)
  2. global_write(a,B(i))
  3. for h=1 to  $\log_2 n$  do
      if ( $i \leq n/2^h$ ) then
        begin
          global_read(B(2i-1), x)
          global_read(B(2i), y)
          z := x + y
          global_write(z, B(i))
        end
      end
end
```

Sum on the PRAM (2)



Key assumptions

Shared memory

- The arrays A and B are stored in the global memory and can be accessed by any processor

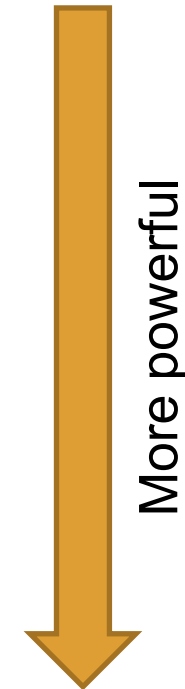
Synchronous mode of operation

- In each unit of time, each processor is allowed to execute an instruction or to stay idle
- Note that condition of “if” statement is satisfied by only some processors. The remaining processors stay idle

PRAM variants



Variant	Description
EREW	Does not allow any simultaneous access to a single memory location
CREW	Simultaneous access for read instructions only
CRCW	• Common: concurrent writes only when all processors write the same value • Arbitrary: arbitrary processor allowed to succeed • Priority: processor with minimum index succeeds



C = Concurrent; E= Exclusive
R- Read; W = Write

Matrix (n×n) multiplication on n^3 processors



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

Input

- A, B in shared memory, $n = 2^k$

Output

- $C=AB$ in shared memory

Algorithm for processor $P_{i,j,l}$

begin

1. $C'(i,j,l) = A(i,l)B(l,j)$

2. **for** $h=1$ **to** $\log n$ **do**

 if $(l \leq n/2^h)$ then

$C'(i,j,l) := C'(i,j,2l-1) + C'(i,j,2l)$

3. **if** $(l=1)$ **then** $C(i,j) := C'(i,j,1)$

end

Requires concurrent read of $A(i,l)$ and $B(l,j) \Rightarrow$ at least CREW



Network model

Computer system modeled as graph $G = (N, E)$

- Each node $i \in N$ represents a processor
- Each edge $(i, j) \in E$ represents a bi-directional communication link

Memory

- Each processor has its own local memory
- No shared memory available

Operation either synchronous or asynchronous

Communication operations



send(X,j) – sends a copy
of X to processor j

receive(Y,i) – blocks until the data from
processor i are received and stored in Y

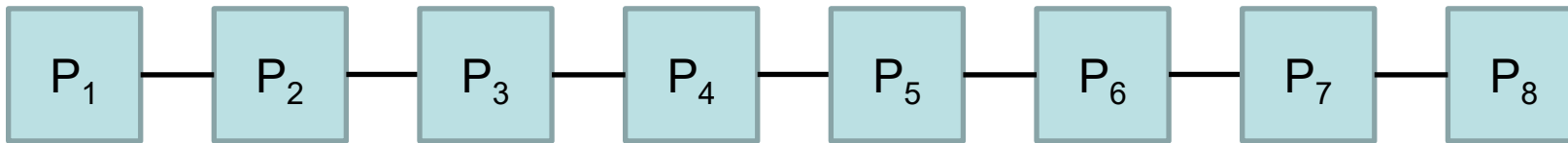
- Processors of an asynchronous network coordinate their activities via messages (**message-passing** model)
 - Communicating processors do not have to be adjacent
 - Process of delivering message from source to destination is called **routing**

Topology

- The network model incorporates the network topology into the model
- Evaluation parameters of the topology
 - **Diameter** – maximum distance between any pair of nodes
 - **Maximum degree** of any node in G
 - **Edge or node connectivity** of G – minimum number of edges / nodes that need to be removed to render network disconnected
 - **Bisection width** – minimum number of edges between nodes that must be removed to cut the network into two roughly equal halves
- Example topologies
 - Linear array, ring, mesh, torus, hypercube, fully connected

Linear processor array and ring

- p processors $P_1, P_2, \dots, P_p$
- Each processor P_i is connected to P_{i-1} and P_{i+1} whenever they exist



- Diameter = $p-1$; maximum degree = 2
- Edge connectivity = 1; bisection width = 1
- A **ring** is a linear array with a wrap-around connection

Matrix–vector product on a ring



- **Given:** $n \times n$ matrix A and vector x of order n
- p processors with $p \leq n$; p divides n evenly with $r = n / p$
- Let A and x be partitioned into p blocks
 - $A = (A_1, A_2, \dots, A_p)$, where each A_i is of size $n \times r$
 - $x = (x_1, x_2, \dots, x_p)$, where each x_i is of size r
- **Output:** $y = Ax = A_1x_1 + A_2x_2 + \dots + A_px_p$



Ring solution

- Let each processor P_i hold initially A_i and x_i
- Each processor can then compute locally product A_ix_i
- Global sum can be calculated by circulating partial sums clockwise

Matrix–vector product on a ring (2)

$$Ax = A_1x_1 + A_2x_2 + \dots + A_px_p$$

Example with $n = 4$ and $p = 2$

$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} \\ a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} \\ a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} \end{bmatrix} \times \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} a_{1,1}x_1 + a_{1,2}x_2 \\ a_{2,1}x_1 + a_{2,2}x_2 \\ a_{3,1}x_1 + a_{3,2}x_2 \\ a_{4,1}x_1 + a_{4,2}x_2 \end{bmatrix} + \begin{bmatrix} a_{1,3}x_3 + a_{1,4}x_4 \\ a_{2,3}x_3 + a_{2,4}x_4 \\ a_{3,3}x_3 + a_{3,4}x_4 \\ a_{4,3}x_3 + a_{4,4}x_4 \end{bmatrix}$$

Matrix–vector product on a ring (3)

Input

- Processor number i
- The number of processors p
- i th submatrix $A_i = A(1 : n, (i-1)r+1 : ir)$
- i th subvector $x_i = x((i-1)r + 1 : ir)$

Output

- Processor p_i computes vector $A_i x_i$ and passes result to the right
- At the end, processor p_1 holds $y = Ax$

begin

1. $z := A_i x_i$
2. **if** $i=1$ **then** $y := 0$ **else** **receive**(y , left)
3. $y := y + z$
4. **send**(y , right)
5. **if** $i=1$ **then** **receive**(y , left)

end

Why?



Discussion

- Computation time: $T_{\text{comp}} = \alpha(n^2/p)$
- Communication time: $T_{\text{comm}} = p \cdot \text{comm}(n)$
 - Each processor has to wait until it receives the intermediate result from its neighbor
 - $\text{comm}(n) = \sigma + n/\tau$
- Total time: $T = T_{\text{comp}} + T_{\text{comm}} = \alpha(n^2/p) + p(\sigma + n\tau)$
 - Tradeoff between computation and communication

σ = latency
 τ = bandwidth

Comparison

Model	Discussion
DAG	• DAG can become quite complex and hard to analyze for realistic problems • Scheduling problem must still be solved • Details of communication and memory allocation missing
PRAM	• Assumes that interprocessor communication is free • Ignores bandwidth limits • Still quite popular - many techniques to handle a variety of computational problems exist
Network	• Difficult to describe and analyze algorithms • Topology restricts portability of algorithms

Parallel performance

- We typically analyze performance for the worst case
- Let Q be a problem for which we have a PRAM algorithm that runs in $T(n)$ time on $P(n)$ processors
- Cost of the algorithm $C(n) = T(n) * P(n)$
- Each parallel algorithm can be converted into a sequential algorithm that runs in $O(C(n))$ time
 - A single processor simulates $P(n)$ processors in $O(P(n))$ time, for each of the $T(n)$ parallel steps
- When the number of processors is larger than $P(n)$, we can achieve the running time using $P(n)$ processors only

Generalization to $p \leq P(n)$ processors

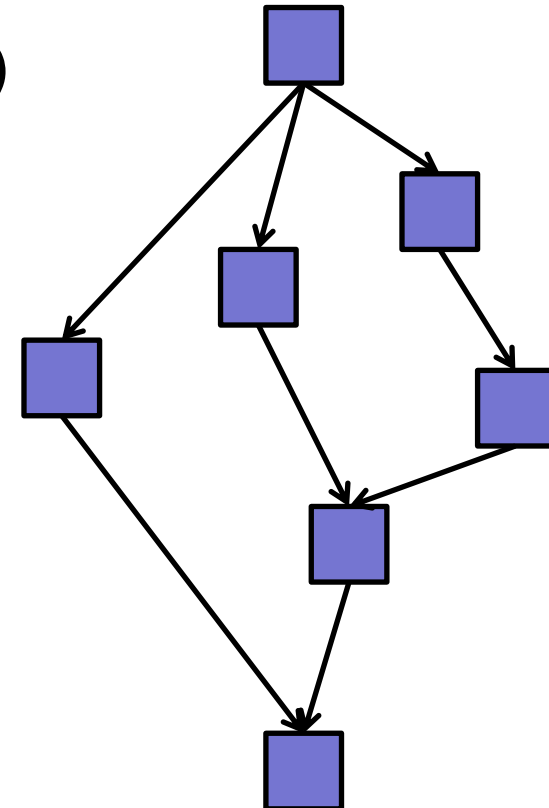
- For each of the $T(n)$ parallel steps, we simulate the $P(n)$ original processors in $O(P(n)/p)$ substeps
 - Takes a total of $O(T(n)*P(n)/p)$ time
- The following ways of measuring performance are asymptotically equivalent
 - $P(n)$ processors and $T(n)$ time
 - $C(n) = P(n)T(n)$ cost and $T(n)$ time
 - $O(T(n)*P(n) / p)$ time for any number of $p \leq P(n)$ processors
 - $O(C(n)/p + T(n))$ time for any number p of processors



Work-depth model

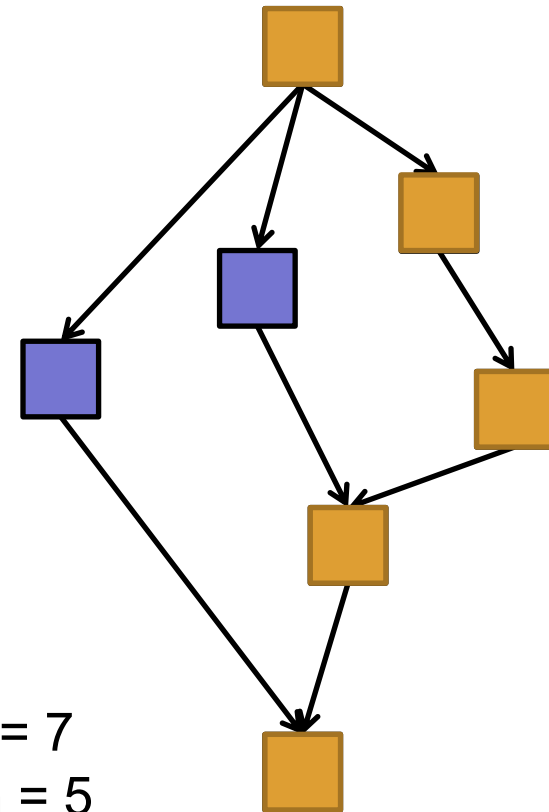
Variation of the DAG model

- Program represented as a **directed acyclic graph** (DAG)
- Hardware consists of workers
- Scheduling is **greedy**
 - No worker idles as long as there is a task ready to run



Work-depth model (2)

- T_p = time to run with p workers
- T_1 = **work**
 - Time for serial execution
 - Accumulated time to complete all tasks
- T_∞ = **depth**
 - Time on ideal machine with infinite #processors
 - Equivalent to length of **critical path**
 - Also known as **span**



Work = 7
Depth = 5

Work-depth model (3)

- No super-linear speedup possible in this model

$$S_p = \frac{T_1}{T_p} \leq \frac{T_1}{T_1/p} = p$$

- Adding processors never slows down an algorithm

$$S_p = \frac{T_1}{T_p} \leq \frac{T_1}{T_\infty} = \frac{\text{work}}{\text{depth}}$$

Average parallelism
in the problem

- But depth influences an algorithm's scalability

Brent's theorem

- Assumption: p less or equal to the number of processors needed to exploit maximum concurrency of the algorithm

$$T_p \leq \frac{(T_1 - T_\infty)}{p} + T_\infty$$

- Intuition – The total work T_1 can be divided into two categories

Perfectly parallelizable

- takes $T_1 - T_\infty$ with single worker
- takes $(T_1 - T_\infty)/p$ with p workers

Imperfectly parallelizable

- takes T_∞



Brent's theorem (2)

- With lower bound

$$\frac{T_1}{p} \leq T_p \leq \frac{(T_1 - T_\infty)}{p} + T_\infty$$

- Estimate of T_p if algorithms offers decent parallelism

$$T_p \approx \frac{T_1}{p} + T_\infty \quad \text{if } T_\infty \ll T_1$$

Speedup bounds

- Lower bound

$$T_p \leq \frac{(T_1 - T_\infty)}{p} + T_\infty$$

$$\Rightarrow S_p \geq \frac{T_1}{\left(\frac{T_1 - T_\infty}{p} + T_\infty\right)}$$

$$\Rightarrow S_p \geq \frac{T_1}{\left(\frac{T_1}{p} + T_\infty\right)}$$

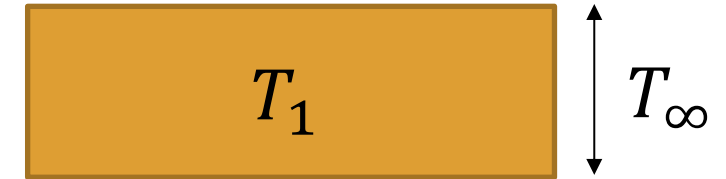
- Upper bound

$$\min\left(p, \frac{T_1}{T_\infty}\right)$$

Over-decomposition

- Greedy scheduling achieves linear speedup if problem is over-decomposed

$$S_p = \frac{T_1}{T_p} \approx p \quad \text{if } p \ll \frac{T_1}{T_\infty}$$



- Excess parallelism is called **parallel slack**

$$\text{Parallel slack} = \frac{S_\infty}{p} = \frac{T_1}{p * T_\infty}$$

Discussion

- Can be used to reason about performance of task-based algorithms
- Cost of computation can be specified in terms of
 - Basic operations of an abstract machine
(e.g., Vector Access Machine “VRAM” [Blelloch, 1990])
 - On the level of programming language constructs (e.g., NESL [Blelloch, 1996])

Discussion (2)

- Comparison to Amdahl's law
 - Also lower bounds for speedup
 - Speedup bounds are usually tighter than with Amdahl's law
 - Notion of serial work generalized to imperfectly parallelizable work
- Simplification
 - Unlimited memory bandwidth
 - Greedy scheduling – violation through lock contention or task pinning

Bulk-synchronous parallel model (BSP)

[Vailant, 1990]

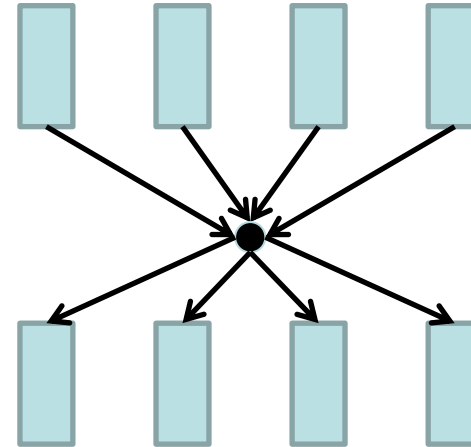


TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

Distributed-memory multiprocessor described in terms of three elements

- Processor/memory modules
- Interconnection network
- A synchronizer which performs barrier synchronization



Computation is a sequence of supersteps

- During each superstep each processor performs local computations and sends and receives messages
 - May only use data available in local memory at the beginning of superstep

LogP model

[Culler et al., 1993]

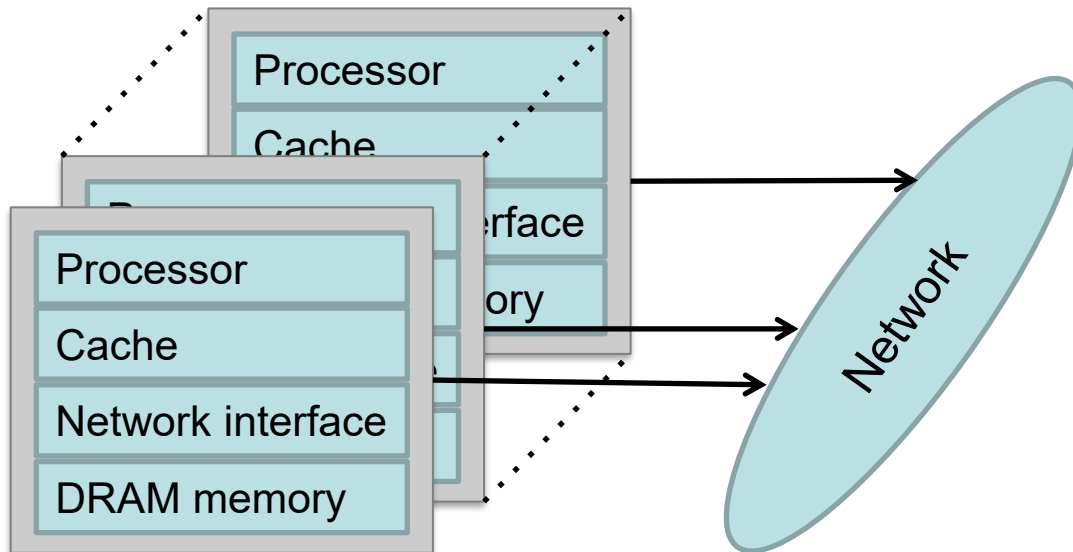


TECHNISCHE
UNIVERSITÄT
DARMSTADT

Parallel
Programming

Designed to reflect general structure of clusters

- Arbitrary number of processing units with distributed memory
- Processing units connected through an abstract communication medium which allows point-to-point communication



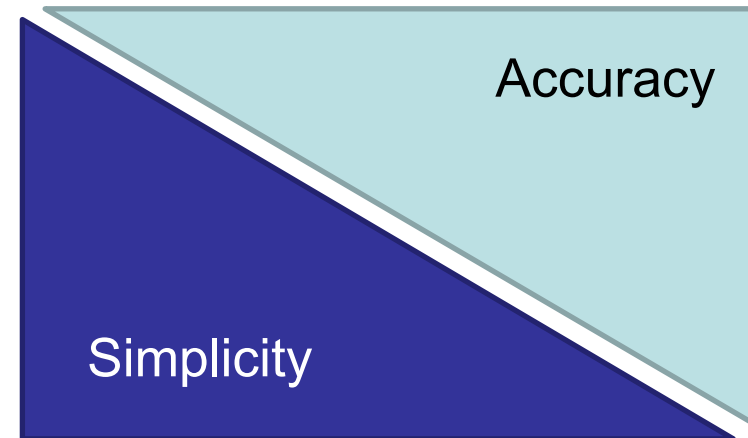
LogP model (2)

Defined by four parameters

L = latency of the communication medium
o = overhead of sending and receiving a message
g = the gap required between two send/receive operations
P = the number of processing units

Summary

- Directed acyclic graph
- Shared-memory model
- Network model
- Work–depth model
- BSP & LogP



Both important for insight